AI Engineering and Claude Architect · Part I, Foundations

Working With Claude, Lesson 5

Prompt Caching and Batches: The Economics

Your finance team asks why the same 50,000-token policy document is billed a thousand times an hour. Your overnight job costs $72,000 a run, and nobody reads the results until morning. Two API features can cut those bills dramatically, if you can avoid the mistakes that silently cancel them.

Module	AI03 of 13
Lesson	5 of 6
Prerequisites	AI02 L5, AI03 L1
Exam relevance	**Batch is direct and heavy** (TS 4.5). Prompt caching appears only as "knowing it exists". Pricing calculations are out of scope
Reading time	~45 minutes

Contents

1. Prompt Caching and Batches: The Economics

Lesson 5 of 6 · Working With Claude

Prompt Caching and Batches: The Economics

Key idea

Prompt caching rewards repetition: identical prefixes are charged at a small fraction of the input price. It works by exact prefix match, so one changing byte near the start silently cancels the saving. **The Message Batches API rewards patience: 50% off, for work that can wait up to 24 hours. Use caching when requests repeat a prefix, batches when nobody is waiting, and both together when both are true.**

> *Before: generation and prefix reuse (AI02 L5), usage fields (AI03 L1). Teaches: how caching works, break-even, breakpoints, silent invalidators, how batches work, SLA calculations, failure handling. Used by: AI05 L3 (Contextual Retrieval), AI08 L6 (async work), AI10 L5 (cost engineering), AI13 L5.*

The hook: two expensive habits

Habit one. Your support assistant sends a 50,000-token policy manual with each request, a thousand times an hour. At Opus 5.5's $4 per million input tokens, that's $0.20 per request, or $200 an hour for reading the same document.

Habit two. A nightly job sends 2 million document analyses at full price. Each has 5,000 input and 800 output tokens, and nobody looks at the results until 9 a.m.

Real output from `code/ai03-claude/.../CacheEconomics.java`:

```
Opus 5.5, 5-minute TTL                without=$ 200.00 with=$ 10.24 saves=94.9% breaks even at 2 requests
Opus 5.5, 1-hour TTL                  without=$ 200.00 with=$ 10.39 saves=94.8% breaks even at 3 requests
Sonnet 4.6-style 0.1x read, 5-minute   without=$ 150.00 with=$ 15.17 saves=89.9% breaks even at 2 requests
2M requests: synchronous=$72,000 batch=$36,000
```

Key idea

About 95% off the repeated prefix, and 50% off the whole overnight run. These figures use the listed multipliers and prices in Anthropic's documentation as of November 2026. They cover only the prefix's cost, and they assume that every request hits the cache.

Intuition: a textbook on your desk, and a night shift

Caching is like keeping the policy manual open on your desk rather than walking to the library each time. You pay a little extra for the first trip, to fetch it and set it up. After that it's nearly free, as long as you keep using it. If you leave it too long, someone tidies it away.

Batches are like a night shift. You leave a stack of work in a tray, and it's done by morning, at half the daytime rate. It's perfect for reports, and useless when a customer is standing at the counter.

Part 1: Prompt caching

The one rule everything follows from

The cache matches on an **exact prefix**. Requests are rendered in the order **tools, then system, then messages**, and any change invalidates everything after it.

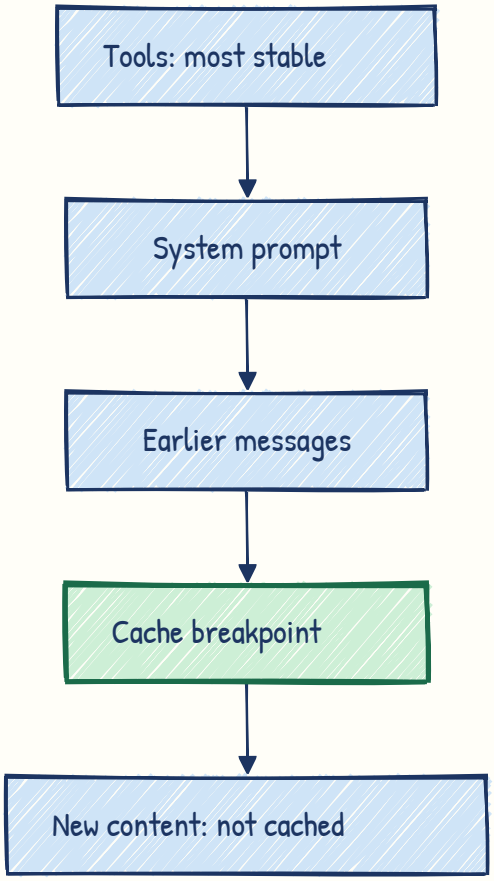


Figure 1. A prefix cache. Stable content goes first. A change at any level invalidates that level and everything after it.

The numbers

From the prompt caching documentation:

Item	Value
Write cost, 5-minute TTL	1.25 × base input price
Write cost, 1-hour TTL	2 × base input price
Read cost, standard	0.1 × base input price
Read cost, Opus 5.5 and Sonnet 5.5	0.05 ×
Read cost, Fable 5.1 and Mythos 5.1	0.025 ×
Default TTL	5 minutes, refreshed free on each use
Maximum breakpoints	4 explicit. Automatic caching uses one of the slots
Lookback per breakpoint	20 blocks
Minimum cacheable length	512 tokens on current models. Up to 4,096 on some older ones

Prompts below the minimum are processed without caching, and no error is returned. That's why you verify using `usage`.

Break-even

Caching costs extra on the first write, so when does it pay?

```
requests=1   uncached=$0.200 cached=$0.250
requests=2   uncached=$0.400 cached=$0.260
requests=3   uncached=$0.600 cached=$0.270
requests=10  uncached=$2.000 cached=$0.340
```

With a 5-minute TTL, **just two requests within the TTL make it worthwhile.** A 1-hour TTL needs three on Opus 5.5, because its write costs double. So the 1-hour option is for traffic where requests are more than 5 minutes apart, or for batches.

Traffic pattern	TTL	Why
Steady, every few minutes or more often	5 minutes	Each read refreshes it for free
Bursts with gaps of 5–60 minutes	1 hour	Otherwise you keep re-writing
One-off	None	The write premium is wasted
Batch jobs	1 hour	Anthropic's docs suggest this, since batches can take longer than 5 minutes

Where to put the breakpoint

Automatic caching is simplest: one top-level `cache_control`, and the breakpoint moves forward as the conversation grows. **Explicit breakpoints** give finer control, for example to cache a large system prompt separately from the conversation.

```
.systemOfTextBlockParams(List.of(
        TextBlockParam.builder()
                .text(policyManual)
                .cacheControl(CacheControlEphemeral.builder().build())
                .build()))
```

Silent invalidators

This is what really goes wrong in production. Nothing fails. Your bill just stays high, because some byte in the prefix changes on every request.

Pattern	Why it breaks caching	Fix
`LocalDateTime.now()` in the system prompt	Prefix changes on every request	Put it in a later message
Request ID or trace ID early in content	Every request is unique	Keep it in metadata, not the prompt
Serializing a `HashMap` to JSON	Key order may vary	Use sorted keys
User name in the system prompt	One prefix per user, so no sharing	Move it to the messages
`if (flag) system += ...`	Each flag combination is a separate prefix	Move flags later
Tools built per user	Tools render first, so nothing is shared	Keep one deterministic tool list, sorted by name
Switching models mid-conversation	Caches are model-scoped	Keep the model stable
Changing `effort` or thinking settings	Invalidates at least the messages level	Keep them stable for a session

```java
// Bad: invalidates the cache on every request.
String system = "Today is " + LocalDate.now() + ". " + policyManual;

// Better: the stable part is cached, and the date comes after the breakpoint.
String dateNote = "Today is " + LocalDate.now() + ".";
```

Verify it, always

```
cache_creation_input_tokens  tokens written
cache_read_input_tokens      tokens read
input_tokens                 tokens after the last breakpoint
```

If `cache_read_input_tokens` stays at zero across repeated requests, something is invalidating your prefix. Make this a dashboard metric: cache hit rate per route.

Concurrency

A cache entry can be used only after the first response begins. If 50 workers start together on an empty cache, all 50 can miss, and each pays to write. So warm the cache with one request, then fan out. AI05's pilot lesson uses this.

Part 2: The Message Batches API

What it is

From the batch processing documentation:

Property	Value
Discount	50% on input and output
Limit per batch	100,000 requests or 256 MB, whichever comes first
Typical time	Most batches finish in under 1 hour
Maximum	Batches expire if processing doesn't finish within 24 hours
Results available	29 days after creation
Order of results	**Any order**
Correlation	`custom_id`
Result types	`succeeded`, `errored`, `canceled`, `expired`

> *The exam's view, and the docs' view*
>
> The docs say most batches finish within an hour. The exam stresses the guarantee: "up to 24-hour processing window, no guaranteed latency SLA". **Both are true, and the second is the one you design around.** "Usually within an hour" can't support a blocking workflow.

The lifecycle

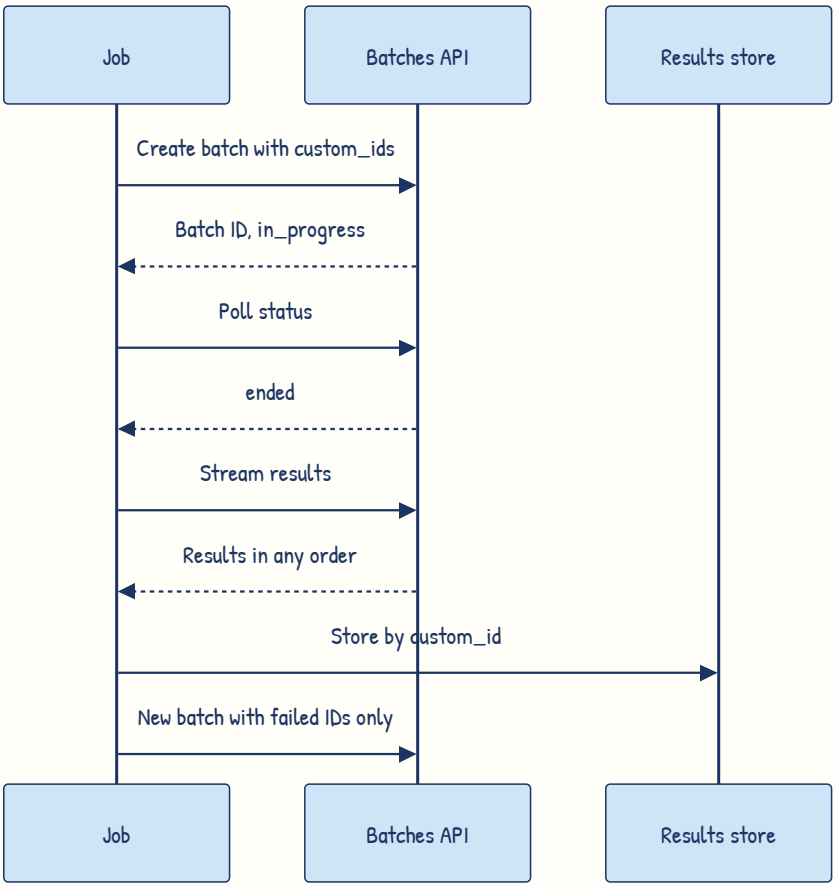


Figure 2. Batch lifecycle. Submit, poll until ended, then read results keyed by custom_id. Only the failures are resubmitted.

Java, from `BatchJobs.java`, compiled against `anthropic-java` 3.2.0, but not run:

```
public String submit(Map<String, String> promptsById) {
    BatchCreateParams.Builder batch = BatchCreateParams.builder();
    promptsById.forEach((id, prompt) -> batch.addRequest(
            BatchCreateParams.Request.builder()
                    .customId(id)
                    .params(BatchCreateParams.Request.Params.builder()
                            .model(model)
                            .maxTokens(4_096L)
                            .addUserMessage(prompt)
                            .build())
                    .build()));
    return client.messages().batches().create(batch.build()).id();
}
```

When batch is right, and when it's wrong

Workload	Use batch?	Why
Overnight technical debt report	**Yes**	Nobody's waiting
Weekly audit	**Yes**	Latency-tolerant
Nightly test generation	**Yes**	Latency-tolerant
Large eval runs	**Yes**	Latency-tolerant
Pre-merge check	**No**	Blocks a developer
Customer chat	**No**	Someone's waiting
An agent that needs tool results	**No, within one request**	See below

The multi-turn limitation

A batched request is sent and answered once. If Claude responds with `tool_use`, you can't return a result into that same request. **The batch API doesn't support multi-turn tool calling within a single request** (TS 4.5). You could run the next turn as a new batch, but each turn could take up to 24 hours, so an agent loop doesn't fit.

Server tools, such as web search, run on Anthropic's side and are supported in batches. The limitation concerns tools **you** have to execute.

SLA arithmetic

The exam guide includes this calculation. If results must be ready within 30 hours of arrival, and a batch can take 24 hours, how often must you submit?

Worst case: a document arrives just after a submission. It waits one full gap, and then takes up to 24 hours.

`gap + 24 ≤ 30`, so `gap ≤ 6 hours`

```
SLA 30h, batch window 24h: submit at least every 6 hours
SLA 26h, batch window 24h: submit at least every 2 hours
```

The guide's example uses 4-hour windows for a 30-hour SLA, which leaves a 2-hour safety margin. Six hours is the theoretical maximum, and a real design keeps a margin. **Expect exam questions to be answerable using this formula, and read what they're asking for: "longest possible" or "safe".**

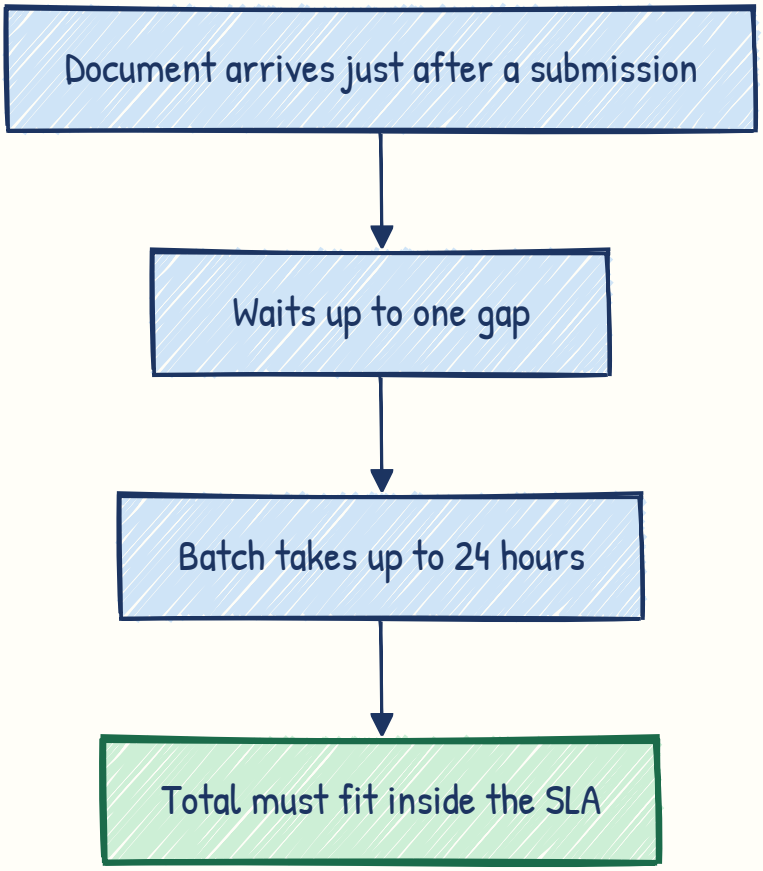


Figure 3. Worst-case timing. The wait for the next submission and the batch window both consume the SLA.

Handling failures

```
public List<String> collect(String batchId, ResultSink sink) {
    List<String> failedIds = new ArrayList<>();
    BatchResultsParams params = BatchResultsParams.builder()
            .messageBatchId(batchId)
            .build();
    try (StreamResponse<MessageBatchIndividualResponse> results =
            client.messages().batches().resultsStreaming(params)) {
        results.stream().forEach(item -> {
            if (item.result().isSucceeded()) {
                String message = item.result().asSucceeded().message().toString();
                sink.accept(item.customId(), message);
            } else {
                failedIds.add(item.customId());
            }
        });
    }
    return failedIds;
}
```

Result	Billed?	What to do
`succeeded`	Yes	Store by `custom_id`
`errored`, invalid request	No	Fix the request. Don't simply resend it
`errored`, server error	No	Resubmit
`expired`	No	Resubmit, and plan your capacity
`canceled`	No	Resubmit if still needed

A document that exceeded the context limit needs **modification**, such as chunking it, rather than a plain resend. TS 4.5 names that example.

Refine on a sample first. A prompt flaw across 100,000 documents is an expensive way to find a bug, so test on a representative sample before batch-processing large volumes. That's TS 4.5 too.

Using both together

Anthropic's documentation says **the discounts from prompt caching and Message Batches can stack**, and suggests a 1-hour TTL for shared context in batches, because a 5-minute entry can expire while a batch runs. Cache hits in batches aren't guaranteed, since requests may process concurrently and in any order, so verify using `usage`.

How Anthropic approaches it

Problem	Anthropic's solution	The trade-off it asks of you
Re-reading long prefixes wastes capacity	Prompt caching	Keep your prefix byte-stable
Cache management is fiddly	Automatic caching	Less fine control
Offline work competes with live traffic	Batches, at 50% off	Accept up to 24 hours
Silent cache misses	Usage fields, and beta cache diagnostics	Monitor them
Long-running shared context	1-hour TTL	Double write cost

This is one of the clearest examples of Anthropic turning a physical fact (AI02 L5) into a contract. It doesn't publish how serving works internally, but it does publish the deal.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: everything synchronous, nothing cached. One path for all work, at full price.

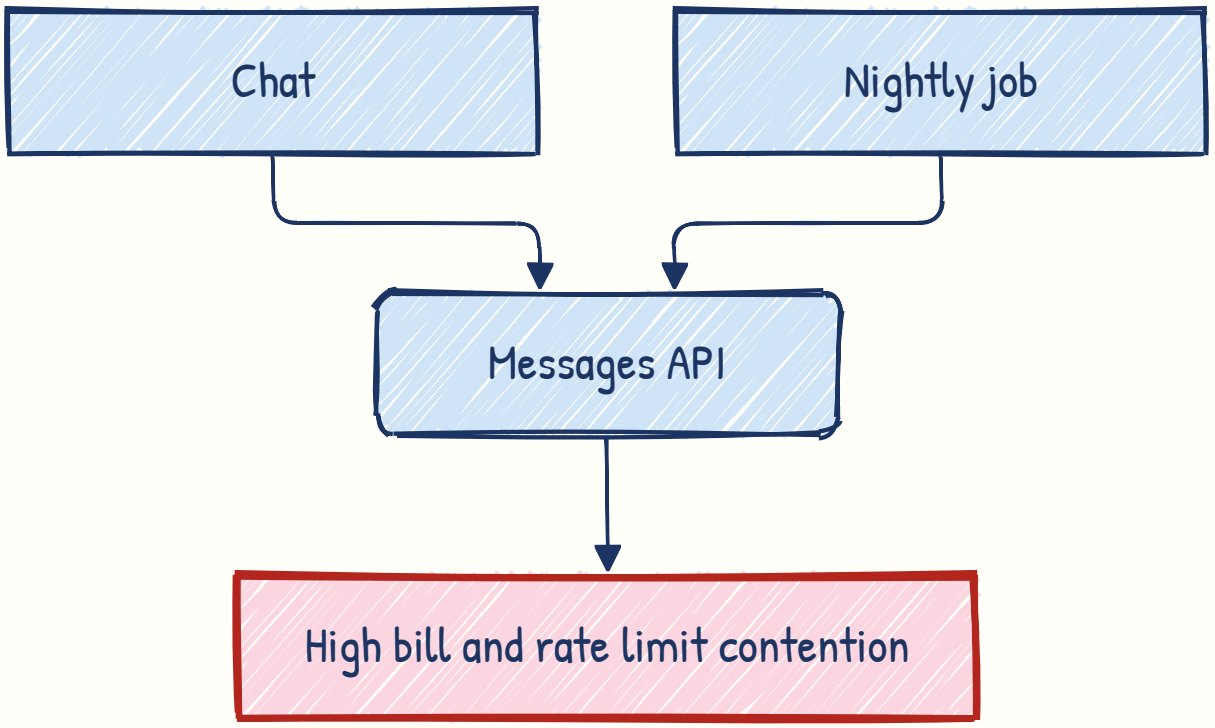


Figure 4. Baseline. Live chat and overnight jobs share one synchronous path, one rate limit, and full prices.

Improved: caching turned on. Prefix reuse helps. New problem: **a silent invalidator can cancel all the savings, and nobody will know, because nothing fails.**

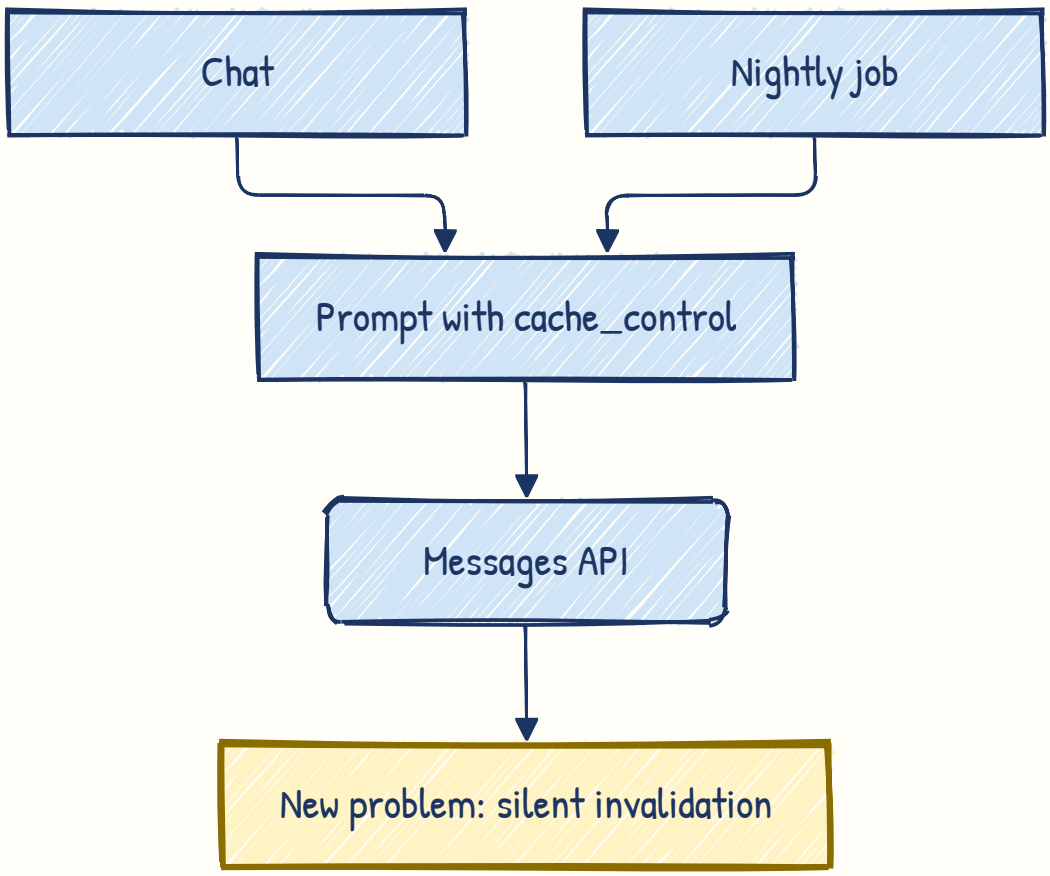


Figure 5. Improved. Caching is enabled, but with no measurement, its benefit can vanish after any innocent-looking prompt change.

Production: separate paths, with measurement.

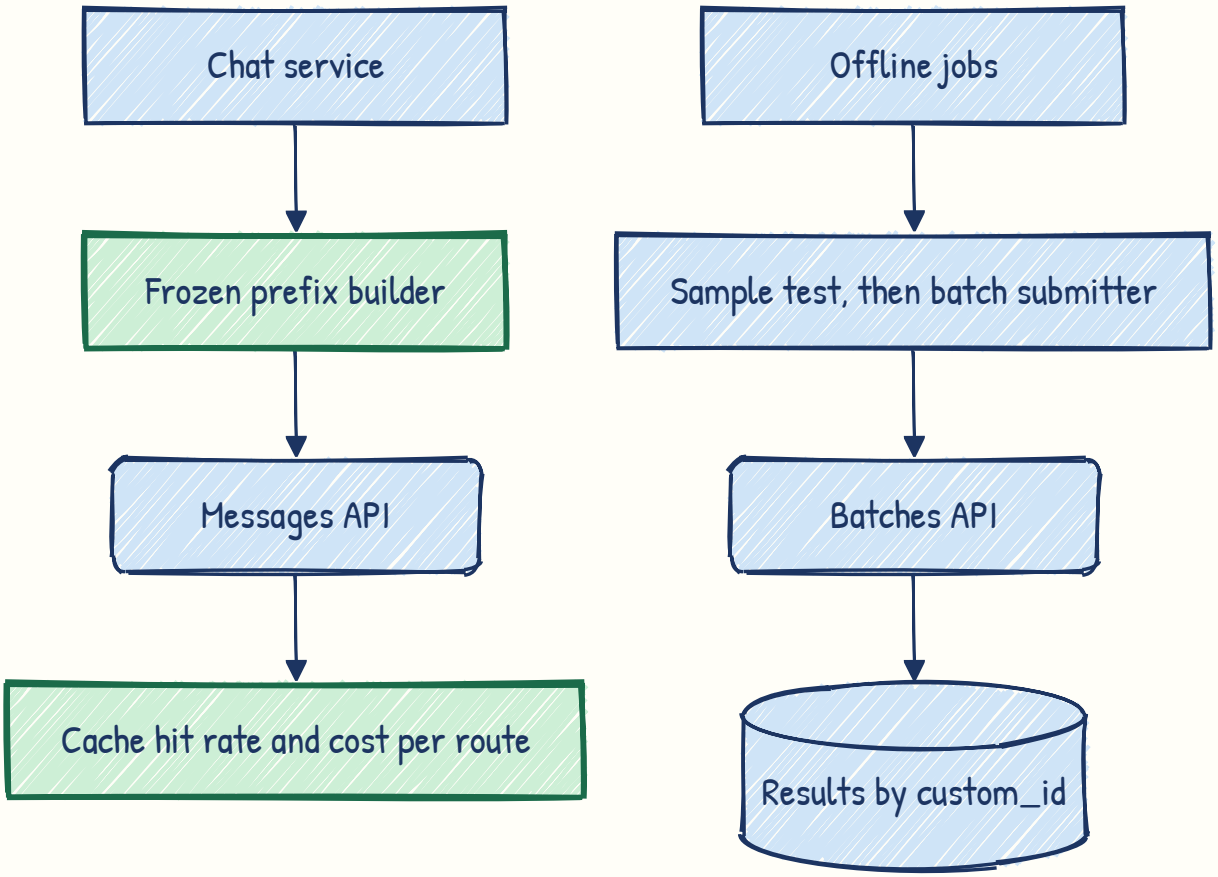


Figure 6. Production. Live traffic uses a frozen prefix, offline work uses batches, and cache hit rate is a monitored metric with an alert.

Component	Why it exists	How it communicates
Frozen prefix builder	Guarantees stable bytes: sorted tools, no timestamps	In-process, with a unit test asserting identical output
Sample test	Catches prompt flaws before a large batch	A small synchronous or batch run
Batch submitter	Schedules within SLA, and resubmits failures	Scheduled job
Results store	Handles any-order results, and idempotent writes	Upsert by `custom_id`
Metrics	Makes invalidation visible	Alert when hit rate falls

Failures

Failure	Effect	Containment
A deploy adds a timestamp to the prefix	Bill jumps, with no errors	Unit test on prefix bytes, plus hit rate alerts
Cold start at peak	Many cache writes	Warm with one request first
Batch expires	Late results, not billed	Monitor, resubmit, and submit earlier
Document exceeds limits	That item errors	Chunk it and resubmit only that ID
Duplicate processing	Double writes	Idempotent upsert by `custom_id`

Trade-offs

Decision	Gain	Cost
5-minute TTL	Cheap writes, free refreshes	Expires in quiet periods
1-hour TTL	Survives gaps	Double write price
Frozen system prompt	High hit rate	Dynamic data must move later
Batch	50% savings	Up to 24 hours, no client tools within a request
Separate paths	Isolation	Two flows to run

When not to use caching: one-off requests, or prefixes below the minimum. **When not to use batch:** anything that blocks a person or a pipeline step, or needs multi-turn tools.

Common mistakes

1. **Putting the date in the system prompt.** It silently cancels caching.
2. **Not checking `cache_read_input_tokens`.** Nothing else reveals a miss.
3. **Fanning out on an empty cache.** Warm it first.
4. **Using batch for pre-merge checks.** There's no latency guarantee.
5. **Matching batch results by position.** Results arrive in any order.
6. **Resubmitting everything after a partial failure.** Use `custom_id`, and resend only failures.
7. **Running 100,000 documents with an untested prompt.** Test a sample first.
8. **Using the 5-minute TTL for long batches.** It can expire before requests run.

What to say in an interview

"For repeated prefixes, I'd use caching and keep the prefix byte-stable, with a hit-rate alert, since a stray timestamp cancels it silently. For latency-tolerant work, I'd use batches at 50% off, submit often enough that gap plus 24 hours fits the SLA, correlate by `custom_id`, and resubmit only failures. Blocking checks stay synchronous."

Certification lens

How this lesson maps to CCAR-F

Direct: TS 4.5 is all of Part 2. **Out of scope:** prompt caching details beyond knowing it exists, and pricing calculations. The SLA calculation **is** in scope, because the guide names it. Don't memorize the caching multipliers for the exam, though you'll use them at work.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Results must be available within 28 hours of a document's arrival, and batch processing can take up to 24 hours. Which submission schedule is the best fit?

A. Every 24 hours.

B. Every 12 hours.

C. Every 8 hours.

D. Every 3 hours.

A **D.** gap + 24 ≤ 28 means gap ≤ 4 hours, and 3 hours fits with a margin. A, B and C can each miss the SLA in the worst case (TS 4.5).

Q Q2 · Exam style, scenario: Structured Data Extraction · Medium. In a batch of 50,000 documents, 400 fail because they exceed the context limit. What's the most efficient response?

A. Resubmit the entire batch with a larger `max_tokens`.

B. Resubmit only the 400, identified by `custom_id`, after chunking them.

C. Resubmit the 400 unchanged, since batch errors are usually transient.

D. Reprocess the 400 through the first 400 result positions.

A **B** (TS 4.5). A wastes money, and `max_tokens` controls output rather than input. C would fail again for the same reason. D is wrong because results don't arrive in request order.

Q Q3 · Exam style, scenario: Multi-Agent Research System · Medium. Choose the **two** statements that are true of the Message Batches API.

A. It offers 50% cost savings.

B. It guarantees completion within one hour.

C. It lets a request run your tools mid-request, and continue.

D. Results are matched to requests using `custom_id`.

E. Results are always returned in request order.

A **A and D.** B: most finish within an hour, but the window is up to 24 hours, with no latency SLA. C: multi-turn tool calling isn't supported within a request. E: results can arrive in any order.

Q Q4 · Calculation · Medium. A 100,000-token prefix on Opus 5.5 ($4 per million input tokens) is used 50 times in 5 minutes. Compute the prefix cost without caching and with caching, using a write of 1.25× and reads of 0.05×.

A Without: 50 × 100,000 × $4 / 1M = **$20.00**. With: one write of $0.50, plus 49 reads at $0.02 each = $0.98, for a total of **$1.48**. That saves about 93%.

Q Q5 · Code analysis · Medium. Why is this cache hit rate zero?

```
Map<String,Object> ctx = new HashMap<>(); ctx.put("requestId", uuid);
system = toJson(ctx) + policyManual;
```

A The request ID differs each time, and it comes before the manual, so the prefix never matches. Remove it from the prompt, or move it after the last breakpoint. Serializing a `HashMap` may also vary key order. Verify the fix using `cache_read_input_tokens`.

Revision - Lesson 5

→ Caching matches an exact prefix, in the order tools, system, messages.

→ Write 1.25× (5-minute) or 2× (1-hour). Read 0.1×, or 0.05× on Opus 5.5 and Sonnet 5.5.

→ TTL is 5 minutes by default, and refreshes free on use.

→ Two requests within the TTL already pay for a 5-minute write.

→ Below the minimum length, no caching happens, and there's no error.

→ Silent invalidators: timestamps, IDs, unsorted JSON, user data, per-user tools, model or effort changes.

→ Verify using cache_read_input_tokens. Warm the cache before fanning out.

→ Batch: 50% off, up to 100,000 requests or 256 MB, up to 24 hours, no latency SLA.

→ Results arrive in any order, so correlate by custom_id. Available for 29 days.

→ No multi-turn client tool calling within a batched request.

→ SLA: gap + 24 hours must fit in the SLA.

→ Resubmit only failures, modifying those that need it. Test prompts on a sample first.

→ Discounts can stack. Use the 1-hour TTL with batches.

Your notes